

SOFTWARE ENGINEERING TERM PROJECT 2

Topic: Testing with NUnit (Automated Unit Testing)

Due Date: 27 / 12 / 2025

General Rules of the Homework

1. Projects will be done in groups of 2-3 students.
2. Assignments will be submitted via UBS system.
3. Report must be in English.
4. All automated tests must be written using the NUnit framework.

1. PROJECT DEFINITION AND PURPOSE

In this project, you will design and implement automated unit tests for the given C# classes. Your goal is to verify the correctness of each class under all relevant scenarios (normal flows, boundary conditions, and invalid inputs) using the NUnit testing framework.

2. SCOPE AND CONSTRAINTS

- Cover all public methods and important branches, including error/exception paths.
- Use meaningful test names and the Arrange-Act-Assert structure.
- Use parameterized tests where they reduce duplication (TestCase).

3. PROVIDED CLASSES

Class 1:

```
public class FizzBuzz
{
    public string GetOutput(int number)
    {
        if ((number % 3 == 0) && (number % 5 == 0))
```

```
        return "FizzBuzz";

    if (number % 3 == 0)
        return "Fizz";

    if (number % 5 == 0)
        return "Buzz";

    return number.ToString();
}
}
```

Class 2:

```
public class DemeritPointsCalculator
{
    private const int SpeedLimit = 65;
    private const int MaxSpeed = 300;

    public int CalculateDemeritPoints(int speed)
    {
        if (speed < 0 || speed > MaxSpeed)
            throw new ArgumentOutOfRangeException();

        if (speed <= SpeedLimit) return 0;

        const int kmPerDemeritPoint = 5;
        var demeritPoints = (speed - SpeedLimit)/kmPerDemeritPoint;
```

```
        return demeritPoints;
    }
}
```

Class 3:

```
public class Stack<T>
{
    private readonly List<T> _list = new List<T>();

    public int Count => _list.Count;

    public void Push(T obj)
    {
        if (obj == null)
            throw new ArgumentNullException();

        _list.Add(obj);
    }

    public T Pop()
    {
        if (_list.Count == 0)
            throw new InvalidOperationException();

        var result = _list[_list.Count - 1];
        _list.RemoveAt(_list.Count - 1);

        return result;
    }
}
```

```
public T Peek()
{
    if (_list.Count == 0)
        throw new InvalidOperationException();

    return _list[_list.Count - 1];
}
```

4. TESTING REQUIREMENTS

- Create a separate test project (e.g., <SolutionName>.Tests)
- For each class, create at least one [TestFixture] and write tests for all public methods.
- Include tests for: typical valid inputs, boundary values, null/empty inputs (if applicable), and invalid inputs that should raise errors or return specific results.

5. PROJECT DELIVERABLES AND EXPECTED DOCUMENTATION CONTENT

- Cover page and brief information about the project must be given.
- A short report (English) including screenshots of your test codes and output showing that all tests pass.
- Each test code must be explained briefly (what is the purpose of doing it).
- All project files and project report pdf file must be in one zip file.

Good luck!